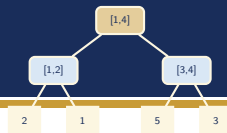


Segment Tree

Fast Range Queries and Dynamic Updates



CSE200 — Data Structures & Algorithms

Query smarter, not harder.

Presented by

Shayan Un Noor 2305024

Md. Shadman Shahriyar Shuvo 2305025

Md. Wasif Haque 2305029

It's 9:31 AM. The stock market just opened,
and 20,000 prices are changing every second —
what's the total value of your portfolio right now?

Recompute everything?

Too slow when this happens thousands of times a second.

Precompute once?

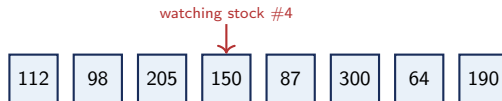
Breaks the moment anything changes.

This is the exact tension every large system — stock feeds, leaderboards, analytics dashboards — fights constantly.

From Wall Street to Array Indices

Translating the problem

Stock i 's price	$A[i]$
Portfolio total	$\text{sum}(l, r)$
Price change	$\text{update}(i, v)$

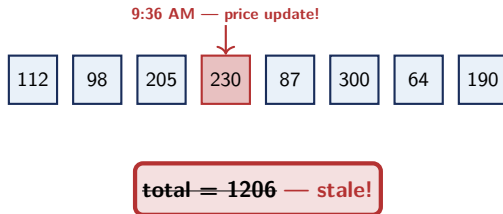


Precomputed total = 1206

From Wall Street to Array Indices

Translating the problem

Stock i 's price	$A[i]$
Portfolio total	$\text{sum}(l, r)$
Price change	$\text{update}(i, v)$



From Wall Street to Array Indices

Translating the problem

Stock i 's price	$A[i]$
Portfolio total	$\text{sum}(l, r)$
Price change	$\text{update}(i, v)$

The catch

Keep the sum fresh $\Rightarrow$ every update is $O(n)$.
Keep updates instant $\Rightarrow$ every query is $O(n)$.

X

Recompute the total
every update $\Rightarrow O(n)$

X

Rescan the range
every query $\Rightarrow O(n)$

What If We Just... Split the Problem?

Instead of touching the whole array every time
— what if we recursively split it in half, solve
each half, and just *combine* the answers?

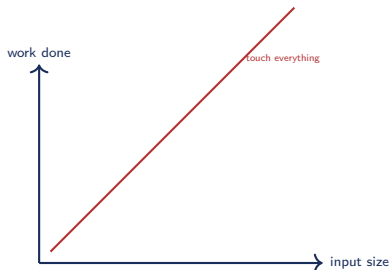
- Each split costs almost nothing.
- Only $O(\log n)$ splits are ever needed to reach any piece.
- Combining is cheap — just merge two summaries.



What If We Just... Split the Problem?

Instead of touching the whole array every time
— what if we recursively split it in half, solve
each half, and just *combine* the answers?

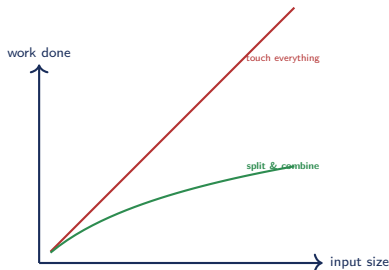
- Each split costs almost nothing.
- Only $O(\log n)$ splits are ever needed to reach any piece.
- Combining is cheap — just merge two summaries.



What If We Just... Split the Problem?

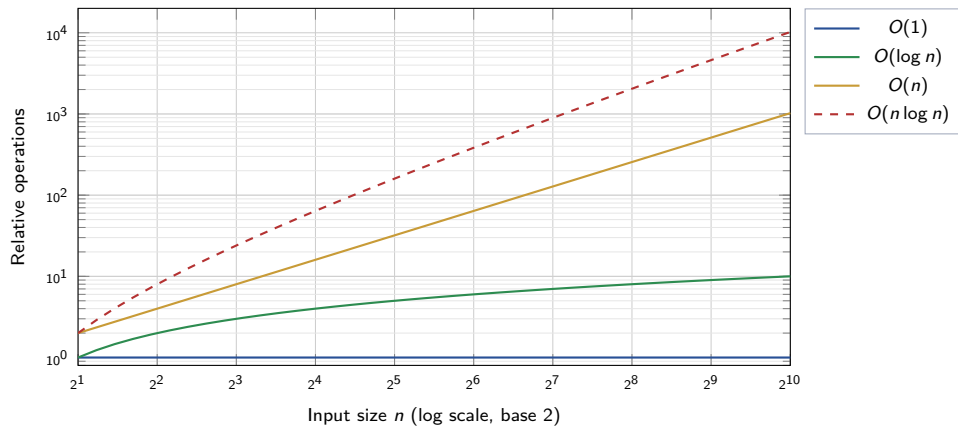
Instead of touching the whole array every time
— what if we recursively split it in half, solve
each half, and just *combine* the answers?

- Each split costs almost nothing.
- Only $O(\log n)$ splits are ever needed to reach any piece.
- Combining is cheap — just merge two summaries.



That gap between the two curves — **that's** the whole idea we're chasing.

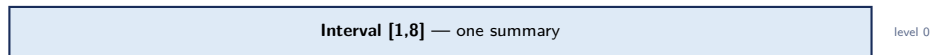
Why $\log n$ Feels Free



$\log_2 n$ grows glacially — e.g. $\log_2(10^9) \approx 30$.

Split It, Store It, Solve It

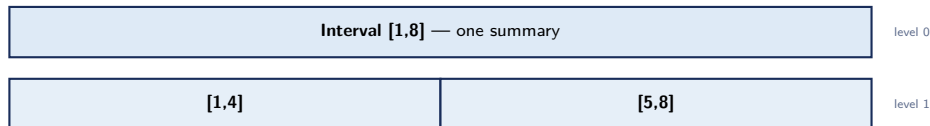
Store a **summary** of each interval at a tree node; queries **glue** summaries together.



- Children split each interval in half; height = $\lceil \log_2 n \rceil$.
- Any range $[l, r]$ decomposes into just $O(\log n)$ pieces.

Split It, Store It, Solve It

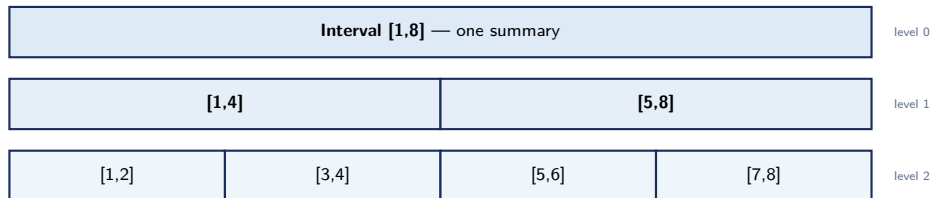
Store a **summary** of each interval at a tree node; queries **glue** summaries together.



- Children split each interval in half; height = $\lceil \log_2 n \rceil$.
- Any range $[l, r]$ decomposes into just $O(\log n)$ pieces.

Split It, Store It, Solve It

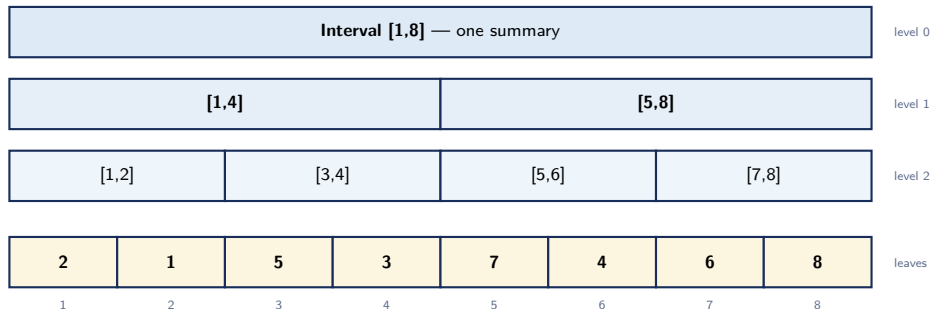
Store a **summary** of each interval at a tree node; queries **glue** summaries together.



- Children split each interval in half; height = $\lceil \log_2 n \rceil$.
- Any range $[l, r]$ decomposes into just $O(\log n)$ pieces.

Split It, Store It, Solve It

Store a **summary** of each interval at a tree node; queries **glue** summaries together.



- Children split each interval in half; height = $\lceil \log_2 n \rceil$.
- Any range $[l, r]$ decomposes into just $O(\log n)$ pieces.

Anatomy of a Segment Tree

[1,8]
36

1

2

3

4

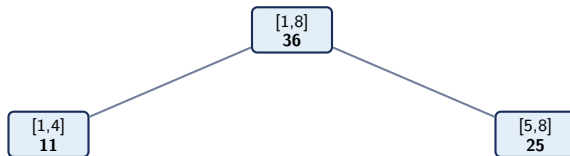
5

6

7

8

Anatomy of a Segment Tree



1

2

3

4

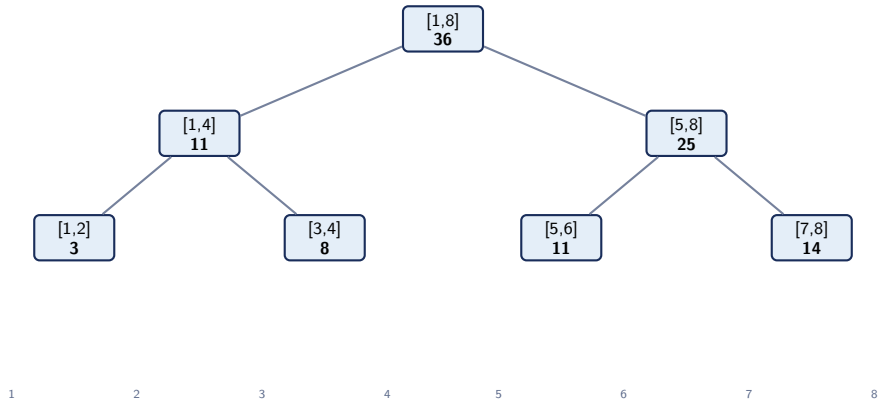
5

6

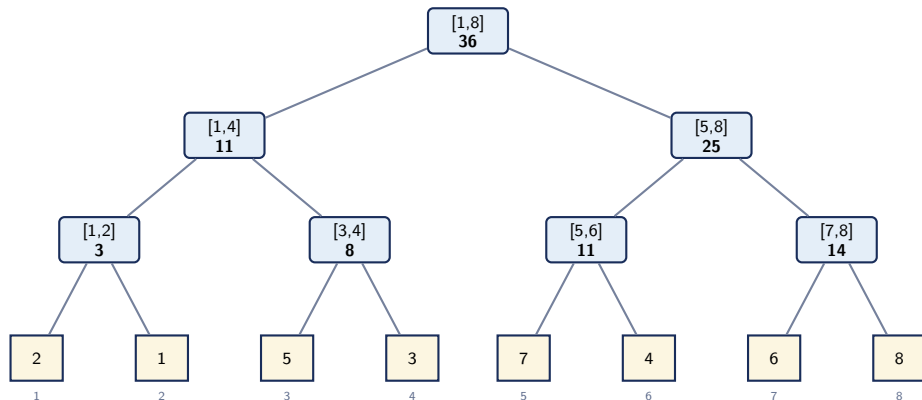
7

8

Anatomy of a Segment Tree



Anatomy of a Segment Tree



Built top-down: each range is recursively **divided** into two smaller halves.

Height = $\lceil \log_2 n \rceil \Rightarrow$ every root-to-leaf path is just $O(\log n)$ long.

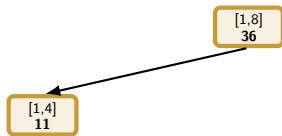
One Change, $\log n$ Fixes: `update(4, 10)`



[1,8]
36

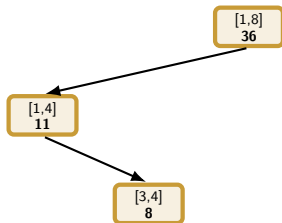
1 2 3 4 5 6 7 8

One Change, $\log n$ Fixes: `update(4, 10)`



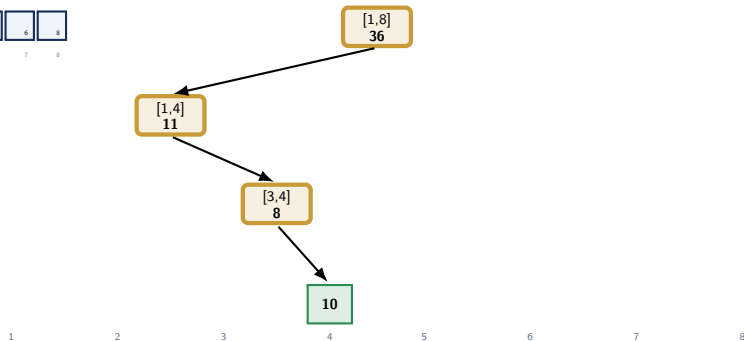
1 2 3 4 5 6 7 8

One Change, $\log n$ Fixes: `update(4, 10)`

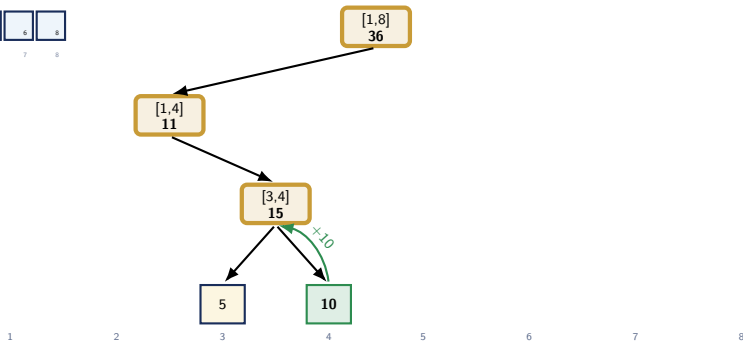


1 2 3 4 5 6 7 8

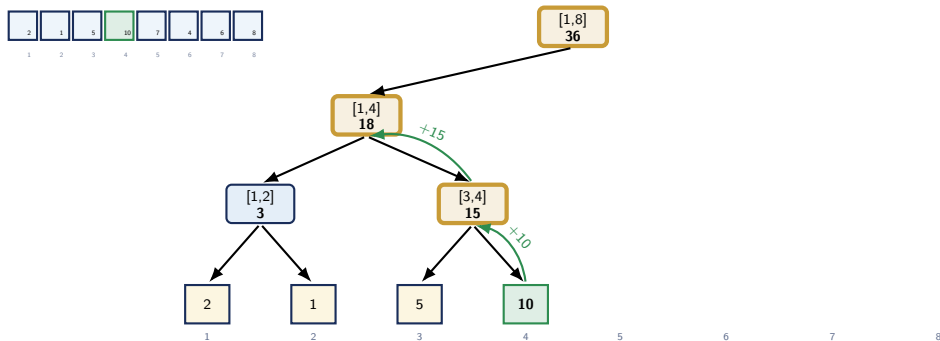
One Change, $\log n$ Fixes: `update(4, 10)`



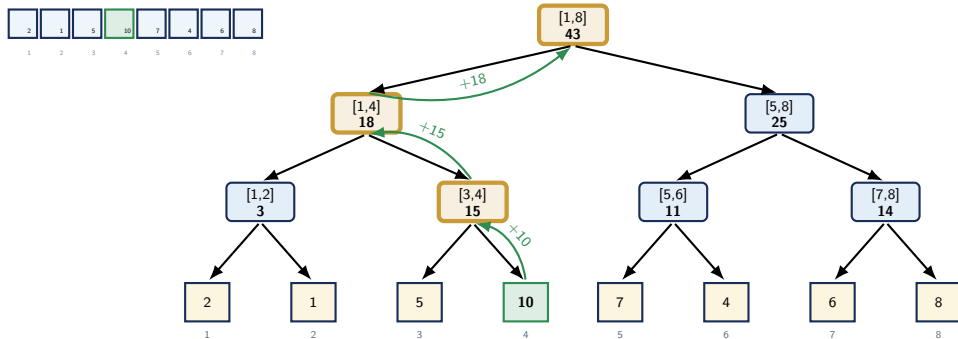
One Change, $\log n$ Fixes: `update(4, 10)`



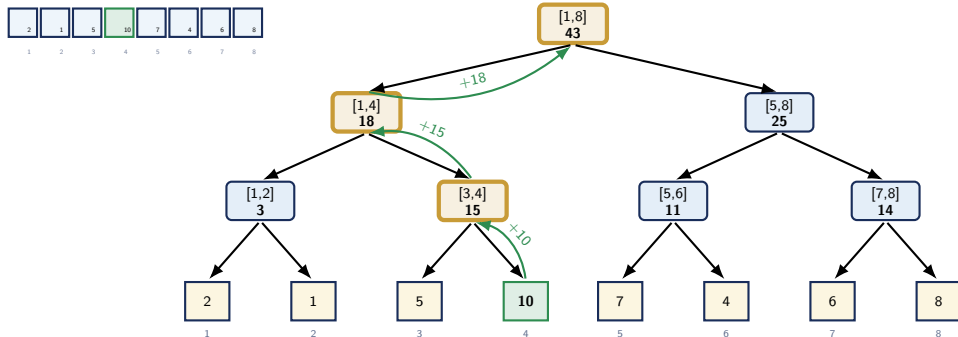
One Change, $\log n$ Fixes: `update(4, 10)`



One Change, $\log n$ Fixes: `update(4, 10)`



One Change, $\log n$ Fixes: `update(4, 10)`

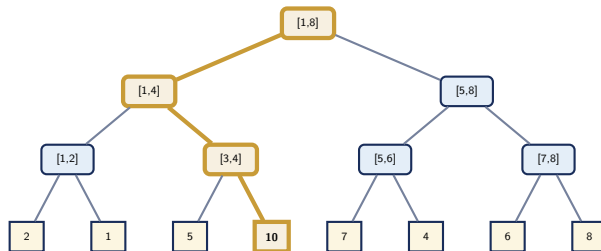


Black = descending to the target index. **Green** = merging back up. Only $\log n$ nodes ever touched.

`update(i, v)` costs exactly $O(\log n)$ — one downward pass, one upward merge

Time Complexity of update()

An update walks exactly **one path**: root $\rightarrow$ target leaf, then merges its way back up along that very same path.



- Going down: one node chosen per level — left *or* right child, never both.
- Coming back up: one merge per level, along that same path.
- Total work $\propto$ number of levels visited = **height of the tree**.

A segment tree built on n leaves is a **full binary tree**, so its height is $\lceil \log_2 n \rceil$.

$$\text{update}(i, v) = O(\log n)$$

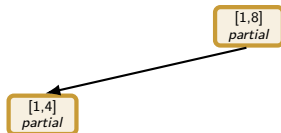
Watching $\text{sum}(3,7)$ Unfold — On the Updated Array



[1,8]
partial

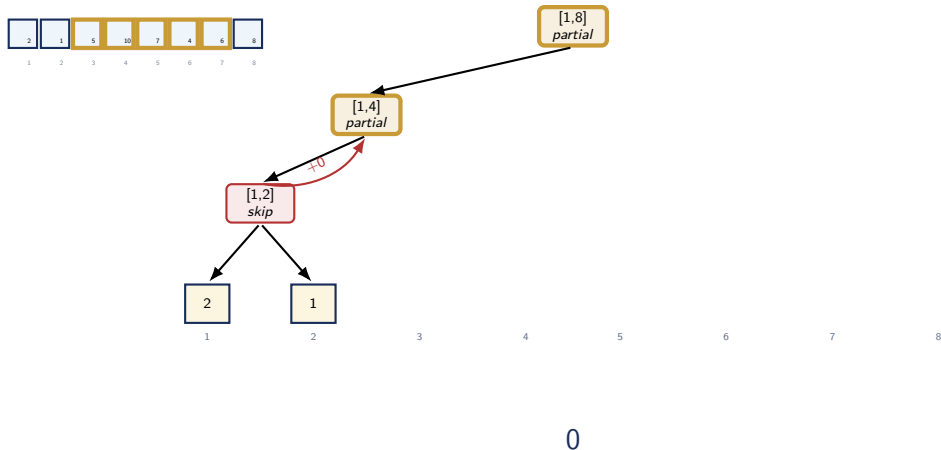
1 2 3 4 5 6 7 8

Watching $\text{sum}(3,7)$ Unfold — On the Updated Array

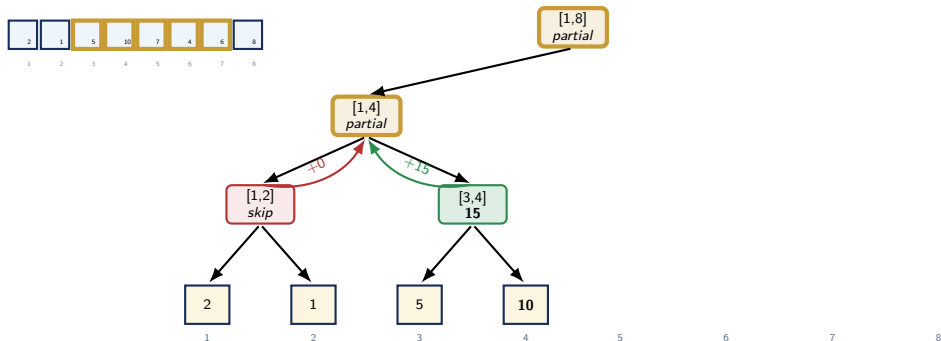


1 2 3 4 5 6 7 8

Watching sum(3,7) Unfold — On the Updated Array

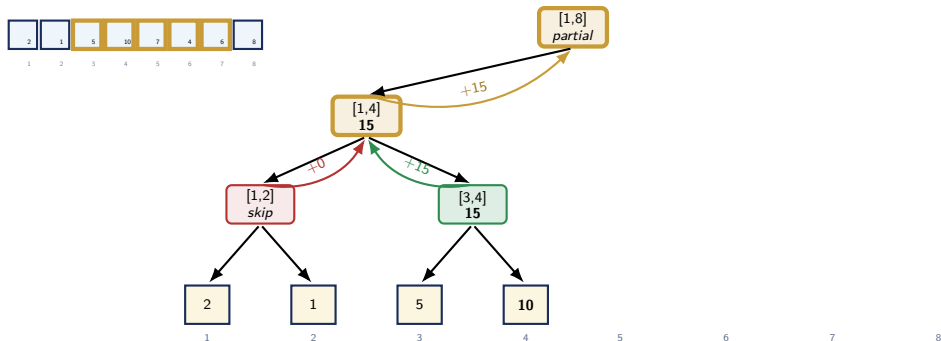


Watching sum(3,7) Unfold — On the Updated Array



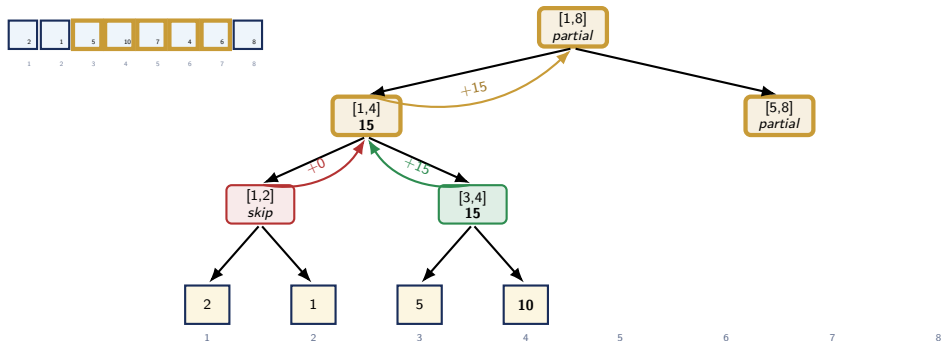
$0 + 15$

Watching sum(3,7) Unfold — On the Updated Array



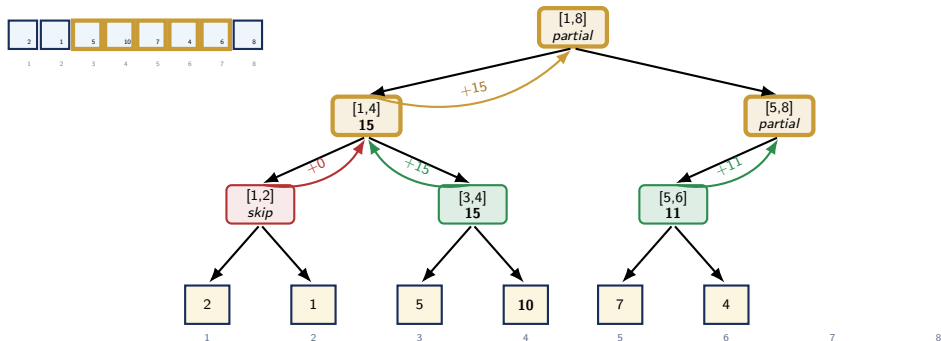
0 + 15

Watching sum(3,7) Unfold — On the Updated Array



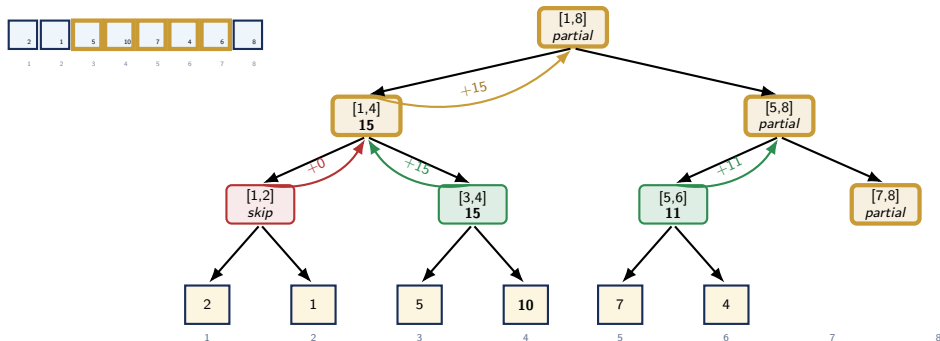
$0 + 15$

Watching sum(3,7) Unfold — On the Updated Array



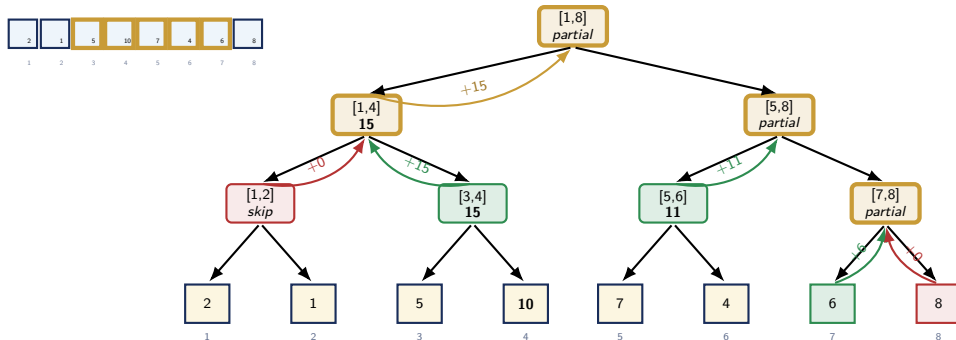
$$0 + 15 + 11$$

Watching sum(3,7) Unfold — On the Updated Array



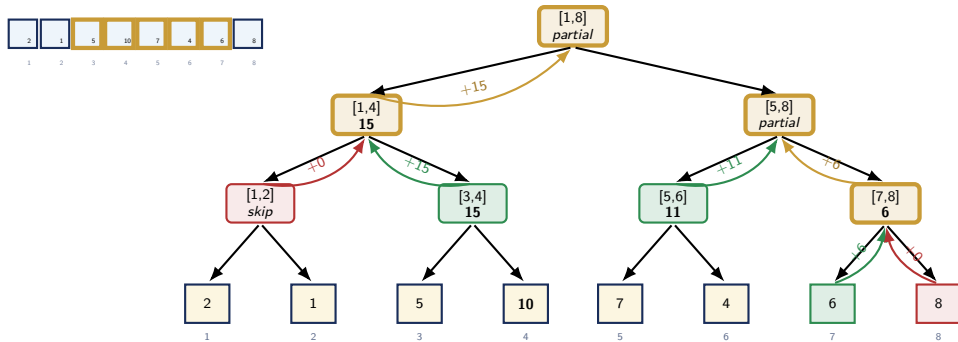
$$0 + 15 + 11$$

Watching sum(3,7) Unfold — On the Updated Array



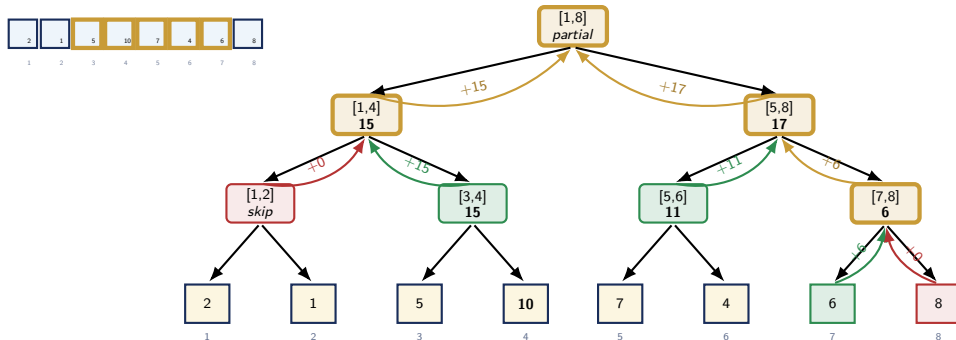
$$0 + 15 + 11 + 6 + 0$$

Watching sum(3,7) Unfold — On the Updated Array



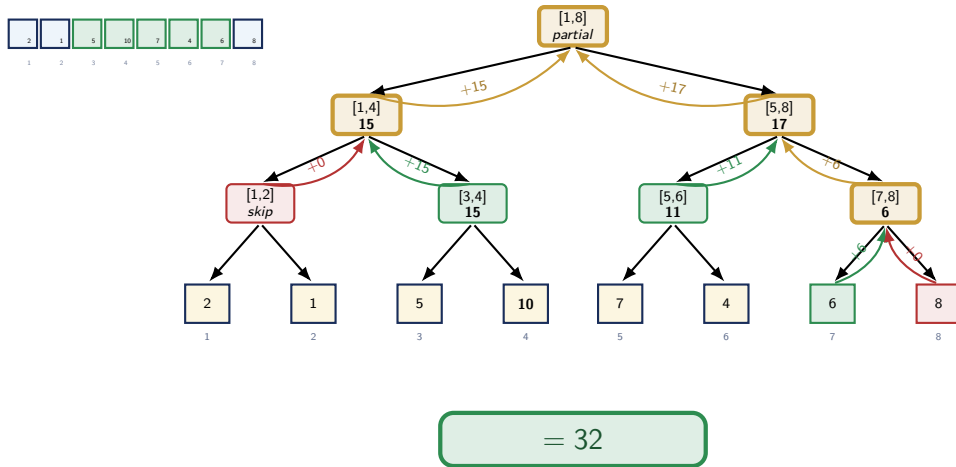
$$0 + 15 + 11 + 6 + 0$$

Watching sum(3,7) Unfold — On the Updated Array



$$0 + 15 + 11 + 6 + 0$$

Watching sum(3,7) Unfold — On the Updated Array



Cells 3–7 in the array above (highlighted green) sum to exactly 32 — confirming the tree's answer. Only 7 of the tree's 15 nodes were ever visited.

Time Complexity of $\text{sum}(l, r)$: The Overlap Argument

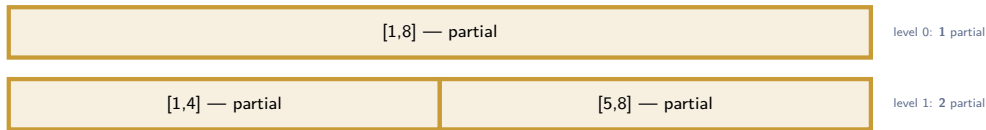
At **every level**, the query boundary can cut through at most **one** segment on the left and **one** on the right — everything else is fully in or fully out.

[1,8] — partial

level 0: 1 partial

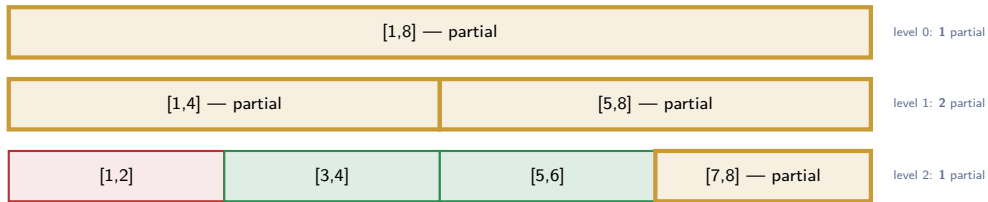
Time Complexity of $\text{sum}(l, r)$: The Overlap Argument

At **every level**, the query boundary can cut through at most **one** segment on the left and **one** on the right — everything else is fully in or fully out.



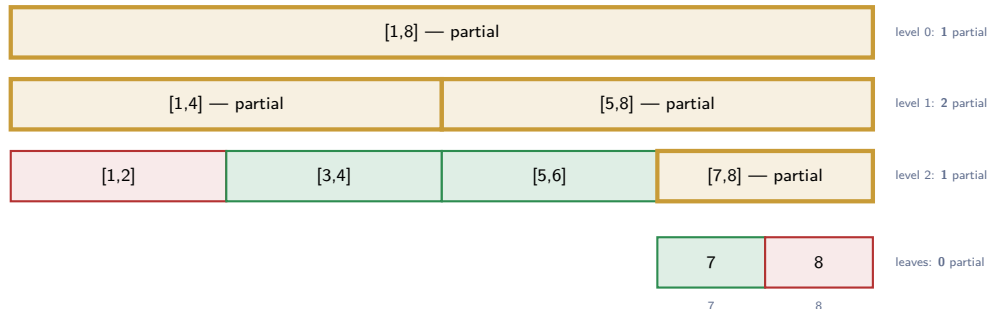
Time Complexity of $\text{sum}(l, r)$: The Overlap Argument

At **every level**, the query boundary can cut through at most **one** segment on the left and **one** on the right — everything else is fully in or fully out.



Time Complexity of $\text{sum}(l, r)$: The Overlap Argument

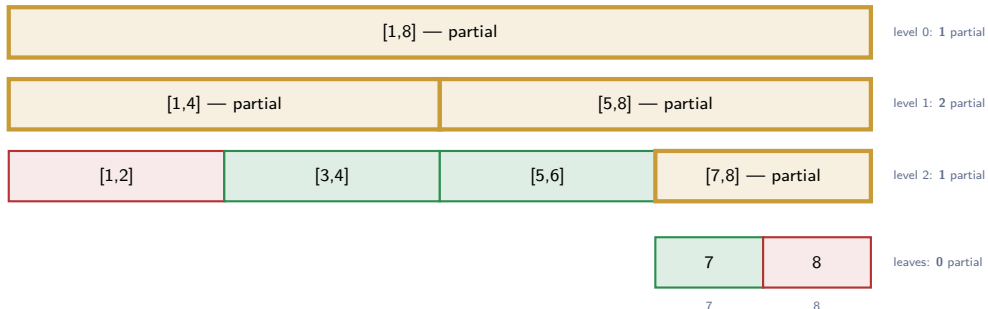
At **every level**, the query boundary can cut through at most **one** segment on the left and **one** on the right — everything else is fully in or fully out.



Green = fully inside (stop, $O(1)$) Red = fully outside (stop, $O(1)$) Gold = partial (only these recurse)

Time Complexity of $\text{sum}(l, r)$: The Overlap Argument

At **every level**, the query boundary can cut through at most **one** segment on the left and **one** on the right — everything else is fully in or fully out.



Green = fully inside (stop, $O(1)$) Red = fully outside (stop, $O(1)$) Gold = partial (only these recurse) At most 2 gold segments per level, and only $O(\log n)$ levels exist:

$$\text{sum}(l, r) = O(\log n)$$

Suppose our system tracks $n = 1024$ instruments and must process $q = 36$ billion operations (queries + updates combined) — assuming ~ 1 nanosecond per elementary step:

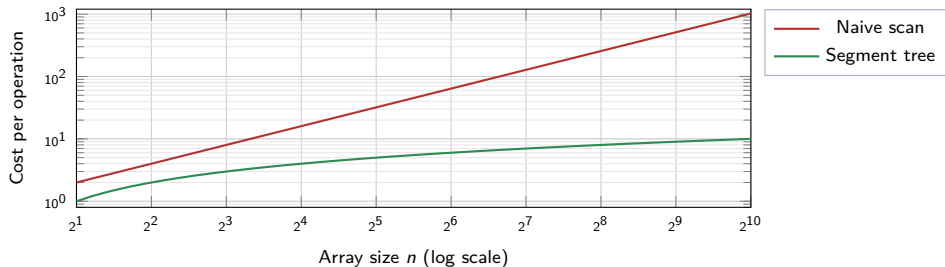
	Naive rescan $O(qn)$	Segment tree $O(q \log n)$
Raw ops	$36 \times 10^9 \times 1024$	$36 \times 10^9 \times 10$
Real time	≈ 10.24 hours	$= 6$ minutes

Suppose our system tracks $n = 1024$ instruments and must process $q = 36$ billion operations (queries + updates combined) — assuming ~ 1 nanosecond per elementary step:

	Naive rescan $O(qn)$	Segment tree $O(q \log n)$
Raw ops	$36 \times 10^9 \times 1024$	$36 \times 10^9 \times 10$
Real time	≈ 10.24 hours	$= 6$ minutes

Naive: start the job, go to work, come home, watch a movie — it's *still running*.
Segment tree: grab a coffee. It's done before the coffee is.

The Complexity Showdown



Structure	Query	Update	Build
Naive array	$O(n)$	$O(1)$	—
Prefix sums	$O(1)$	$O(n)$	$O(n)$
Fenwick tree	$O(\log n)$	$O(\log n)$	$O(n \log n)$
Segment tree	$O(\log n)$	$O(\log n)$	$O(n)$

At $n = 10^6$: naive $\approx 10^6$ ops/query — segment tree ≈ 20 . That's the whole pitch.

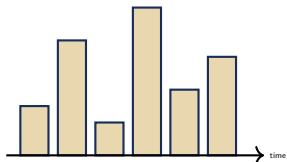
Same Tree, Different Rules

Problem	Node stores	Combine rule
Range sum	running total	add
Range minimum	smallest value	take min
Range maximum	largest value	take max
Range GCD	gcd so far	gcd
...any associative rule	...any summary	...just swap it in

Why this matters

The **tree** never changes — only what a node stores and how two nodes combine. Master one structure, unlock a dozen problems.

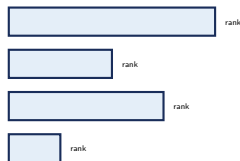
Recap — **Part II**: build once in $O(n)$, then update and query any range, with any associative rule, in $O(\log n)$ each.



Time-series & stock feeds

range-sum/min/max over any time window, instantly

Where does this actually run?



Live leaderboards

k -th largest score & rank updates in $O(\log n)$

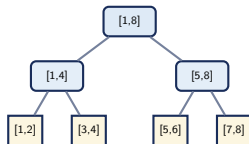


Booking & scheduling systems
find/reserve a free range of seats or slots
fast

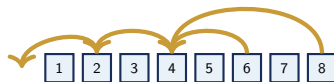
The common thread

Any system that needs “**summarize a range, then change part of it**” — repeatedly, at scale — is a segment tree in disguise.

Segment Tree vs Fenwick: Pick Your Fighter



Segment Tree



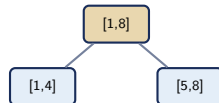
Fenwick Tree (BIT)

	Segment Tree	Fenwick Tree
Query / Update	$O(\log n)$	$O(\log n)$
Memory	$\sim 4n$	$\sim n$
Range updates (lazy)	✓ natural	✗ awkward
Flexibility (min/max/gcd/custom)	✓ any associative op	✗ sum-like only
Code footprint	larger	tiny

Fenwick wins on simplicity & memory; segment tree wins on flexibility & power. Pick based on what the problem demands.

Takeaways

- ✓ Segment trees answer **range queries** and apply **updates**, both in $O(\log n)$.
- ✓ One structure, many rules: sum, min, max, gcd, or any associative merge.
- ✓ Powers real systems: finance, leaderboards, scheduling, and more.



References

CLRS, *Introduction to Algorithms* — Competitive Programmer's Handbook (Laaksonen) — cp-algorithms.com, *Segment Tree*